

1. Difference between nil and false

nil represents the absence of a value or nothing exists, while false represents a boolean value that explicitly means not true; both are treated as falsy in Ruby conditions, but nil is commonly used to indicate missing or undefined data, whereas false is used when a condition is evaluated and results in false.

2. Symbols vs Strings

Strings are mutable objects, meaning their content can be changed after creation and each string creates a new object in memory, whereas symbols are immutable, unique, and stored only once in memory, which makes them faster and more memory-efficient, so symbols are commonly used as hash keys, method names, and identifiers.

3. Blocks vs Procs vs Lambdas

Blocks are anonymous pieces of code passed to methods and are not objects, Procs are objects that wrap blocks and can be stored, reused, and passed around, and lambdas are a special type of Proc that enforce strict argument checking and have different return behavior, where return inside a Proc exits the enclosing method but return inside a lambda exits only the lambda itself.

4. How does Ruby garbage collection work?

Ruby uses automatic garbage collection to manage memory by tracking object references, marking objects that are still reachable, and removing unused objects from memory using a mark-and-sweep algorithm with generational garbage collection to improve performance and reduce memory leaks.

5. What is duck typing?

Duck typing in Ruby means that an object's suitability is determined by the methods it responds to rather than its class, so if an object behaves like the expected type by implementing required methods, it can be used regardless of its actual class.

6. Explain method lookup path

When a method is called in Ruby, Ruby searches for it first in the object's class, then in included modules (in reverse order), then in the superclass, continuing up the inheritance chain until it reaches `BasicObject`, and raises a `NoMethodError` if the method is not found.

7. What are modules used for?

Modules are used in Ruby to group related methods and constants, provide reusable shared behavior through mixins, avoid multiple inheritance issues, and create namespaces to prevent method and class name collisions.

8. Include vs Extend

`include` mixes a module's methods as instance methods into a class so all objects can use them, while `extend` adds the module's methods as class-level methods, affecting only the class itself and not its instances.

9. What is self in different contexts?

In Ruby, `self` refers to the current execution context, meaning inside a class it refers to the class itself, inside an instance method it refers to the object, inside a module it depends on how the module is used, and in the top-level context it refers to the main object.

10. How does Ruby handle exceptions?

Ruby handles exceptions using `begin`, `rescue`, `else`, and `ensure` blocks, where errors are raised as objects of the `Exception` class, `rescue` is used to handle errors, and `ensure` always executes regardless of whether an exception occurs.

11. What is monkey patching?

Monkey patching is the practice of reopening and modifying existing Ruby classes or modules at runtime to add or change behavior, which is powerful and flexible but risky because it can cause unexpected side effects across the application.

12. What are refinements?

Refinements are a Ruby feature that allows scoped monkey patching by limiting method changes to a specific file or module, which helps avoid global side effects and makes code safer and more maintainable.

13. How does Ruby handle threads?

Ruby supports native threads that run concurrently and share the same memory space, allowing efficient I/O-bound operations, while thread scheduling and context switching are managed by the Ruby interpreter.

14. What is GIL?

The Global Interpreter Lock (GIL) ensures that only one thread executes Ruby bytecode at a time, which simplifies memory management and prevents race conditions but limits true parallel execution for CPU-bound tasks.

15. Mutable vs immutable objects

Mutable objects can be changed after creation, such as strings and arrays, while immutable objects cannot be modified once created, such as symbols, integers, and frozen objects, which helps ensure data safety and predictability.

16. How does freeze work?

The freeze method makes an object immutable by preventing any changes to its state, and any attempt to modify a frozen object raises a runtime error, which is useful for protecting constants and shared data.

17. Explain yield

`yield` is used inside a method to transfer control to the block passed to that method, allowing the method to execute custom logic defined by the caller and making methods more flexible and reusable.

18. What are enumerators?

Enumerators are Ruby objects that allow iteration over collections without requiring a block, enable lazy evaluation, and make it possible to chain multiple iteration methods efficiently.

19. Difference between map, each, reduce

`each` is used for iteration and returns the original collection, `map` transforms each element and returns a new array, and `reduce` accumulates values into a single result by applying an operation across the collection.

20. How does Ruby handle memory?

Ruby automatically handles memory management by allocating objects on the heap, tracking references, and using garbage collection to free unused objects, so developers do not need to manually allocate or deallocate memory.

21. What is Rails philosophy?

Rails is built with the philosophy of **Convention over Configuration (CoC)** and **Don't Repeat Yourself (DRY)**, meaning it provides sensible defaults so developers write less configuration code, and it avoids code duplication by organizing logic consistently across the application.

22. MVC flow in Rails request lifecycle

In the Rails MVC flow, a request goes from the **router** to a **controller action**, the controller may interact with **models** to fetch or change data,

and then the controller uses a **view** to generate the HTML or API response that is sent back to the client.

23. What happens when you hit a Rails URL?

When you hit a Rails URL, the web server receives the HTTP request, Rails passes it into the **Rack interface**, the request travels through the **middleware stack**, Rails matches the URL to a route, then executes the corresponding controller action, renders a view or returns data, and finally sends the HTTP response back through middleware to the client.

24. What is Rack?

Rack is a **Ruby webserver interface** that defines a minimal API for processing HTTP requests and responses, allowing Rails (and other Ruby frameworks) to work with many different web servers in a consistent way.

25. Middleware stack explanation

The middleware stack is a sequence of components that process requests and responses between the web server and Rails application, handling tasks like logging, session management, security, caching, and more, where each middleware can inspect or modify the request/response before the next one runs.

26. How does routing work?

Routing in Rails uses definitions in `config/routes.rb` to match incoming HTTP requests (URL + verb) to specific **controller actions**, passing any route parameters to the controller so it can handle the request.

27. RESTful routing principles

RESTful routing means designing routes around resources using standard HTTP verbs like GET, POST, PUT/PATCH, and DELETE, where each verb maps to a conventional controller action such as `index`, `show`, `create`, `update`, and `destroy`, making APIs predictable and consistent.

28. Strong parameters

Strong parameters are a Rails mechanism that requires developers to **explicitly specify which request parameters are allowed** to be assigned to models, helping prevent unwanted or malicious data from being saved.

29. Mass assignment protection

Mass assignment protection refers to protecting model attributes from being updated in bulk without intention; Rails handles this using strong parameters so only permitted attributes can be assigned when creating or updating records.

30. How does Rails handle sessions?

Rails handles sessions by storing a session identifier or encrypted session data in a **cookie** on the client, and on each request Rails reads that data to maintain persistent user state such as login information across requests.

31. Cookies vs sessions

Cookies store small bits of data **on the client's browser**, while sessions store user state that Rails can manage and secure — often stored on the server or encrypted in cookies — making sessions more suitable for sensitive or long-lived data.

32. CSRF protection mechanism

Rails enables **Cross-Site Request Forgery (CSRF) protection by default**, generating a unique token for each session that must be included with non-GET form submissions or AJAX requests and verified on the server to ensure the request came from the legitimate site.

33. What is authenticity token?

An authenticity token is a random, unique token that Rails embeds in forms or request headers, tied to the session, which Rails verifies on the server for non-GET requests to prevent CSRF attacks.

34. What are concerns?

In Rails, concerns are modules used to share reusable logic between models or controllers, helping keep code organized and avoiding duplication without complex inheritance.

35. When to use concerns?

You use concerns when you have common functionality shared across multiple models or controllers, such as shared validations, scopes, callbacks, or helper methods, so that DRY principles are followed.

36. What is Zeitwerk?

Zeitwerk is the **modern code loader in Rails** that automatically loads classes and modules based on file names and directory structure, making autoloading and reloading thread-safe and consistent with conventions.

37. How does autoloading work?

Rails autoloading loads classes or modules **on demand when they are first referenced**, relying on consistent file naming and directory structure so that constants map directly to files without needing manual require statements.

38. Eager load vs lazy load

Lazy loading loads code only when it is first used (fast boot and dev mode), while eager loading loads all application code at startup (good for production performance and thread safety), with Zeitwerk supporting both in a consistent way.

39. Rails environments differences

Rails environments like **development, test, and production** have different settings: development reloads code on each request and shows detailed

errors; test isolates data and behavior for running tests; production caches classes and optimizes performance and security.

40. Initializers role

Initializers are Ruby files in `config/initializers` that run when the app starts, setting up configuration, middleware, libraries, and any custom startup logic before the app begins processing requests.

41. Explain Active Record pattern

The Active Record pattern is an ORM design pattern where each database table is represented by a Ruby class and each row is represented by an object, allowing developers to interact with the database using Ruby methods instead of writing SQL directly, while combining data access and business logic in one place.

42. N+1 query problem and solution

The N+1 query problem occurs when Rails runs one query to fetch parent records and then runs an additional query for each associated record, causing performance issues, and it is solved by eager loading associations using methods like `includes`, `preload`, or `eager_load`.

43. includes vs joins vs preload vs eager_load

`joins` performs SQL joins without loading associated objects and is mainly used for filtering, `includes` automatically decides whether to use separate queries or joins to avoid N+1 queries, `preload` always loads associations using separate queries, and `eager_load` forces a LEFT OUTER JOIN and loads everything in a single query.

44. Callbacks - good and bad use cases

Callbacks are methods that run automatically at specific lifecycle events like before save or after commit, and they are useful for simple tasks like data normalization or logging, but they should be avoided for complex business logic because they make code harder to debug, test, and maintain.

45. Validations vs database constraints

Validations are application-level checks that ensure data correctness before saving records, while database constraints like NOT NULL, UNIQUE, and foreign keys enforce data integrity at the database level, and both should be used together for maximum safety.

46. Transactions usage

Transactions group multiple database operations into a single unit of work so that either all changes succeed or none are applied, ensuring data consistency, especially when updating multiple records or performing critical financial or state-changing operations.

47. Locking - optimistic vs pessimistic

Optimistic locking assumes conflicts are rare and checks for data changes using a version column before updating, while pessimistic locking locks records at the database level to prevent concurrent updates, making optimistic locking better for performance and pessimistic locking safer for high-conflict scenarios.

48. Polymorphic associations

Polymorphic associations allow a model to belong to more than one other model using a single association by storing both an ID and a type column, which is useful when multiple models share a common relationship like comments or attachments.

49. STI vs polymorphic

Single Table Inheritance (STI) stores multiple related subclasses in a single table using a type column, while polymorphic associations allow a model to belong to multiple unrelated models, making STI better for shared behavior and polymorphic associations better for flexible relationships.

50. How migrations work?

Migrations are Ruby classes that manage database schema changes over time, allowing developers to create, modify, and rollback tables and columns in a consistent and version-controlled way across environments.

51. Reversible migrations

Reversible migrations allow Rails to automatically undo schema changes when rolling back by using `change` instead of separate `up` and `down` methods, as long as the operations are reversible.

52. How to handle large tables migrations?

Large table migrations should be handled carefully by avoiding table locks, using background migrations, batching data updates, adding indexes concurrently, and deploying schema changes separately from data changes to prevent downtime.

53. Indexing strategies

Indexing improves query performance by allowing the database to find records faster, and good strategies include indexing foreign keys, frequently searched columns, unique fields, and composite indexes for multi-column queries while avoiding over-indexing.

54. UUID vs integer primary keys

Integer primary keys are smaller and faster for indexing, while UUIDs provide global uniqueness and are better for distributed systems, but UUIDs increase index size and can impact performance if not used carefully.

55. How does Rails handle connections?

Rails manages database connections using a connection pool, where each thread checks out a connection when needed and returns it after use, ensuring efficient and safe database access in multi-threaded environments.

56. Connection pool tuning

Connection pool tuning involves configuring the pool size and timeout to match the number of application threads and database capacity, preventing connection exhaustion while maintaining optimal performance.

57. How to debug slow queries?

Slow queries can be debugged by enabling query logs, using tools like EXPLAIN, analyzing slow query logs, checking N+1 issues, and adding proper indexes to reduce query execution time.

58. Explain scope

A scope is a reusable and chainable query defined in a model that encapsulates common query logic, making database queries cleaner, more readable, and consistent across the application.

59. When not to use scopes?

Scopes should not be used when logic is complex, depends on runtime conditions, or performs side effects, because scopes should remain simple, predictable, and focused only on querying data.

60. How do you write raw SQL safely?

Raw SQL should be written safely using parameterized queries, bound variables, or Active Record sanitization methods to prevent SQL injection and ensure secure interaction with the database.

61. Fat model vs service objects

A fat model means putting a lot of business logic inside Active Record models, which makes models very large and difficult to understand, test, and maintain. Service objects move complex business logic into separate Ruby classes, keeping models focused mainly on database interactions and making the codebase cleaner and easier to manage.

62. When to use service objects?

Service objects should be used when the logic is complex, involves many steps, touches multiple models, or calls external services like APIs. They help keep controllers thin, models simple, and make the logic easier to test and reuse.

63. What is a form object?

A form object is a plain Ruby class that handles form input, validations, and saving data when a single form works with multiple models or complex rules. It helps keep controllers small and avoids putting too much form-related logic into models.

64. What is presenter / decorator?

A presenter or decorator is used to handle view-related logic such as formatting data, combining fields, or deciding how values are displayed on the UI. This keeps models focused on business rules and keeps views clean and readable.

65. PORO in Rails

PORO stands for Plain Old Ruby Object and refers to a normal Ruby class that does not depend on Rails features. POROs are used to write business logic, calculations, or helpers in a simple and reusable way that is easy to test.

66. How to organize business logic?

Business logic should be organized by responsibility, where models handle data and validations, controllers handle HTTP requests, and complex workflows are placed in service objects or POROs. This separation makes the code easier to read, test, and maintain.

67. Rails concerns misuse examples

Concerns are misused when they contain complex business rules, heavy database queries, or workflow logic, because this hides important behavior inside modules. Overusing concerns can make the code hard to trace and debug.

68. Interactors vs services

Interactors are a structured type of service object designed to handle one specific action with a clear input and output. Service objects are more flexible and can be written in many styles, making them easier to adapt but less standardized.

69. Background jobs architecture

Background jobs allow time-consuming tasks to run outside the web request cycle using a job queue and worker processes. This improves application performance and user experience by keeping web requests fast.

70. Active Job vs Sidekiq

Active Job provides a common interface in Rails for writing background jobs, while Sidekiq is a powerful background job processor that executes those jobs using Redis. Sidekiq is often used as the backend for Active Job in production.

71. Idempotency in background jobs

Idempotency means a background job can run multiple times without creating duplicate or incorrect results. This is important because jobs can fail and retry, and idempotent jobs help keep data consistent.

72. How to handle retries?

Retries should be handled by setting retry limits, adding delay between attempts, logging failures, and designing jobs so repeated execution does not break data. This helps ensure stability during temporary failures.

73. How do you handle race conditions?

Race conditions happen when multiple processes update the same data at the same time, causing inconsistent results. They are handled using database transactions, locking mechanisms, and unique constraints to ensure data safety.

74. How do you implement audit logs?

Audit logs are implemented by recording important changes such as who updated a record, what was changed, and when it happened. This can be done using callbacks, service-level logging, or dedicated audit tables.

75. Multi-tenant architecture in Rails

Multi-tenant architecture allows a single Rails application to serve multiple customers while keeping their data separate and secure. This is commonly achieved using tenant IDs, separate schemas, or separate databases.

76. How do you build REST APIs in Rails?

REST APIs in Rails are built using controllers that respond with JSON instead of HTML, following REST principles like using proper HTTP verbs and status codes. Rails provides strong routing, serializers, and middleware support to build clean, scalable APIs.

77. JSON serialization approaches

JSON serialization converts Ruby objects into JSON responses for APIs. Rails supports different approaches like serializers, JBuilder templates, or `to_json`, each helping control which attributes and relationships are exposed.

78. ActiveModelSerializers vs JBuilder

ActiveModelSerializers use Ruby classes to define how objects are serialized, making responses reusable and structured, while JBuilder uses

view templates to manually build JSON, giving more control but requiring more code.

79. API versioning strategies

API versioning allows changes without breaking existing clients and is commonly done using URL paths, request headers, or subdomains. Versioning helps maintain backward compatibility while adding new features.

80. Authentication strategies (Devise, JWT)

Devise is used for session-based authentication with cookies, mainly for web apps, while JWT is token-based authentication commonly used for APIs. JWT allows stateless authentication, making APIs scalable and suitable for mobile clients.

81. Authorization - Pundit vs CanCanCan

Pundit uses simple Ruby policy classes for authorization and follows a strict, explicit approach, while CanCanCan uses a centralized ability file with more magic. Pundit is preferred for clarity, while CanCanCan is quicker to set up.

82. How to secure APIs?

APIs are secured by using authentication, authorization, HTTPS, strong parameters, rate limiting, input validation, and proper error handling. Security headers and token expiration also help prevent misuse.

83. Rate limiting strategies

Rate limiting restricts how many requests a client can make in a given time window to prevent abuse. It is commonly implemented using middleware, Redis counters, or API gateways.

84. SQL injection prevention

Rails prevents SQL injection by using parameterized queries and Active Record query methods. Developers should avoid string interpolation in SQL and always use bindings or sanitization methods.

85. XSS protection in Rails

Rails protects against XSS attacks by escaping HTML output by default in views. Using safe helpers and avoiding raw HTML rendering helps prevent malicious scripts from running in the browser.

86. Caching types in Rails

Rails supports multiple caching types such as page caching, fragment caching, low-level caching, and HTTP caching. Caching improves performance by reducing database queries and expensive computations.

87. Fragment caching

Fragment caching stores parts of a view that do not change often, so Rails can reuse them instead of rendering every time. This helps reduce rendering time and database access.

88. Russian doll caching

Russian doll caching nests cached fragments inside other cached fragments, allowing only changed parts to be re-rendered. It works well with models that have `updated_at` timestamps.

89. HTTP caching headers

HTTP caching uses headers like ETag, Cache-Control, and Last-Modified to let browsers or proxies cache responses. This reduces server load and improves response time.

90. How to optimize Rails performance?

Rails performance is optimized by reducing N+1 queries, adding proper indexes, using caching, background jobs, eager loading, and profiling slow code. Monitoring tools help identify bottlenecks.

91. How do you monitor Rails apps?

Rails apps are monitored using logging, performance monitoring tools, error tracking services, and metrics dashboards. Monitoring helps detect slow requests, errors, and memory issues early.

92. Memory leak debugging

Memory leaks are debugged by tracking memory usage over time, analyzing object growth, checking background jobs, and using profiling tools. Leaks often come from long-running processes or retained objects.

93. How do you handle file uploads?

File uploads in Rails are handled using multipart forms and storage services. Files can be stored locally or uploaded to cloud storage like AWS, with validations for size and type.

94. Active Storage internals

Active Storage manages file uploads by storing file metadata in the database and actual files in a storage service. It supports direct uploads, variants, and multiple storage backends.

95. WebSockets with ActionCable

ActionCable enables real-time features in Rails using WebSockets, allowing the server to push updates to clients. It is commonly used for chat, notifications, and live updates.

96. When not to use ActionCable?

ActionCable should not be used for simple updates or high-scale broadcasting, as it consumes server resources. External services are better for heavy real-time workloads.

97. How do you test Rails apps?

Rails apps are tested using unit tests, integration tests, and system tests to verify models, controllers, APIs, and user flows. Automated tests help ensure code quality and prevent regressions.

98. RSpec vs Minitest

RSpec provides a readable, behavior-driven syntax and rich features, while Minitest is lightweight and faster with a simpler style. RSpec is popular for clarity, while Minitest is closer to Ruby core.

99. How do you test background jobs?

Background jobs are tested by checking job enqueueing, execution, and side effects using test adapters. Jobs can be run inline or mocked to ensure correct behavior.

100. How do you deploy Rails apps?

Rails apps are deployed by setting up servers, configuring environments, running migrations, precompiling assets, and using tools like CI/CD pipelines. Deployment ensures the app runs reliably in production.